PROQRAMLAŞDIRMANIN ƏSASLARI · EV TAPŞIRIĞI

Jafil & Shavi

Sayta Edilən Əlavələrin Tam İzahı

HTML · CSS · JavaScript — hər əlavə, hər funksiya, hər yeni anlayış

20 yeni xüsusiyyət | 17 JavaScript funksiyası | Müəllim sualları və cavabları

Mündəricat

1. Ümumi baxış — nə əlavə olundu?
2. HTML əlavələri
3. CSS əlavələri
4. JavaScript — hər funksiyanın izahı
5. Əsas anlayışlar lüğəti
6. Müəllimin verə biləcəyi suallar və cavablar

1 Ümumi baxış — nə əlavə olundu?

Saytdakı **6 məhsul olduğu kimi qaldı** (adları, şəkilləri, qiymətləri dəyişmədi). Bütün əlavələr saytı daha rahat, gözəl və "canlı" etmək üçündür. Köhnə fayllar `kohne-versiya/` qovluğunda saxlanılıb ki, müqayisə edə biləsən.

#	Əlavə	Harada?	Nə edir?
1	**Loqo yolunun düzəldilməsi**	HTML	Loqo başqa kompüterdə də görünür
2	**Yeni fontlar (Inter + Montserrat)**	HTML+CSS	"ə" və digər Azərbaycan hərfləri düzgün görünür
3	**İşlək səbət paneli**	HTML+CSS+JS	Məhsullar siyahısı, say (+/−), silmək, cəm qiymət
4	**Səbəti yadda saxlama**	JS	Səhifə yenilənəndə səbət itmir (localStorage)
5	**Bildiriş (toast)**	HTML+CSS+JS	Aşağıda "səbətə əlavə olundu" mesajı çıxır
6	**Məhsul axtarışı**	HTML+JS	Yazdıqca məhsullar süzülür
7	**Sıralama**	HTML+JS	Qiymətə və ada görə düzülüş
8	**Şəkli böyütmə (modal)**	HTML+CSS+JS	Məhsul şəklinə klikləyəndə böyük açılır
9	**Yazı effekti**	HTML+CSS+JS	Hero-da sözlər hərf-hərf yazılıb silinir
10	**"Niyə Biz?" bölməsi**	HTML+CSS	3 üstünlük kartı
11	**Telefon və WhatsApp**	HTML+CSS	055 867 23 27 — bir kliklə zəng və ya WhatsApp
12	**Əlaqə formu + e-poçt**	HTML+CSS+JS	Formu yoxlayır və mesajı e-poçta göndərir
13	**Footer + avtomatik il**	HTML+CSS+JS	Səhifənin sonu, il avtomatik yazılır
14	**Sürüşdürmə animasiyası**	CSS+JS	Kartlar ekrana girəndə aşağıdan üzə çıxır
15	**Kiçilən header**	CSS+JS	Aşağı düşəndə yuxarı panel yığcamlaşır
16	**"Yuxarı qalx" düyməsi**	HTML+CSS+JS	Bir kliklə səhifənin başına qayıdır
17	**Escape düyməsi**	JS	Açıq pəncərələri klaviatura ilə bağlayır
18	**Kənarlarda bitki naxışı**	SVG+CSS	Hər iki kənarda sarmaşıq, yarpaqları küləkdə tərpənir
19	**Animasiyalar**	CSS	Menyu, səbət titrəməsi, hover effektləri
20	**Mobil uyğunluq**	CSS	Telefonda hər şey səliqəli görünür

> **Üç faylın rolu (qısa xatırlatma):**
> **HTML** — saytın *skeleti*: nə var (düymə, şəkil, form). **CSS** — saytın *görünüşü*: rəng, ölçü, animasiya.
> **JavaScript** — saytın *beyni*: kliklədikdə nə baş verir, hesablamalar, yaddaş.

2 HTML əlavələri

2.1 Loqo yolunun düzəldilməsi

HTML

```
<!-- KÖHNƏ (səhv) -->
<img src="C:\Users\ASUS\Desktop\jafil-shavi-website\jafil.jpeg">

<!-- YENİ (düzgün) -->
<img src="jafil.jpeg" alt="Jafil & Shavi Loqo" class="sayt-loqosu">
```

Köhnə versiyada **mütləq yol** (absolute path) yazılmışdı — yəni sənin kompüterindəki tam ünvan. Sayt başqa kompüterdə və ya internetdə açılanda bu qovluq olmadığı üçün loqo görünməzdi. **Nisbi yol** (relative path) — `jafil.jpeg` — deməkdir ki, "şəkil HTML faylının yanındadır". Bu, hər yerdə işləyir.

2.2 Yeni fontlar — Azərbaycan hərfləri üçün

HTML

```
<!-- KÖHNƏ: Syne və Plus Jakarta Sans "ə" hərfini düzgün göstərmirdi -->
<!-- YENİ: Inter və Montserrat Azərbaycan hərflərini tam dəstəkləyir -->
<link href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;500;600;700;800&family=Montserrat:wght@600;700;800&display=swap"
rel="stylesheet">
```

Hər fontun içində yalnız müəyyən hərflərin şəkli (**qlif**) olur. Köhnə fontlarda **"ə" hərfi yox idi**, ona görə brauzer onu başqa fontdan götürürdü və hərf sözün içində yad, səliqəsiz görünürdü. **Inter** (mətn üçün) və **Montserrat** (başlıqlar üçün) fontlarında bütün Azərbaycan hərfləri var: ə, ğ, ı, İ, ö, ü, ç, ş. `wght@400;...;800` — fontun hansı qalınlıqlarını yükləyəcəyimiz; `display=swap` — font yüklənənə qədər mətn sistem fontu ilə görünür.

CSS

```
body {
    font-family: 'Inter', 'Segoe UI', Roboto, Arial, sans-serif;
}

h1, h2, h3 {
    font-family: 'Montserrat', 'Inter', 'Segoe UI', sans-serif;
}
```

`font-family`-də bir neçə font vergüllə yazılır — bu, **ehtiyat siyahısı**dır: birinci yüklənməsə, ikincisi istifadə olunur və s.

2.3 Kliklənən səbət

HTML

```
<div class="sebet-qutusu" id="sebetQutusu" onclick="sebetiAcBagla()">
    🛒 Səbət (<span id="sebet-sayi">0</span>)
</div>
```

`onclick` — **hadisə atributu**dur: elementə kliklәyəndə dırnaq içindəki JavaScript funksiyası işə düşür. `id` isə elementin unikal adıdır; JavaScript-də `document.getElementById("sebetQutusu")` ilə həmin elementi tapırıq.

2.4 Hero-da yazı effekti sahəsi

HTML

```
<p class="yazi-effekti"><span id="yaziSahesi"></span><span class="kursor">|</span></p>
```

Birinci `span` boşdur — içini JavaScript doldurur. İkinci `span` yanıb-sönən kursordur (“|”), animasiyası CSS-dədir. `span` — sətir daxili (inline) elementdir, yəni yeni sətirdən başlamır.

2.5 “Niyə Biz?” bölməsi

HTML

```
<section class="ustunlukler gorunme" id="ustunlukler">
    <div class="ustunluk-karti">
        <div class="ustunluk-ikona">🎯</div>
        <h3>Yüksək Dəqiqlik</h3>
        <p>Hər detal millimetr dəqiqliyi ilə çap olunur.</p>
    </div>
    ... (daha 2 kart)
</section>
```

`section` — **semantik** teqdir: səhifənin mənalı bir bölməsini göstərir. Bir elementə bir neçə klass vermək olar: `class="ustunlukler gorunme"` — burada `ustunlukler` görünüş üçündür, `gorunme` isə sürüşdürmə animasiyası üçün. Menyuya da `<a href="#ustunlukler">` linki əlavə olundu — `#` işarəsi həmin `id`-li yerə atlayır.

2.6 Axtarış və sıralama paneli

HTML

```
<div class="alet-paneli">
    <input type="text" id="axtarisXanasi" placeholder="🔍 Məhsul axtar..."
oninput="mehsullariAxtar()">
    <select id="siralamaSecimi" onchange="mehsullariSirala()">
        <option value="standart">Sıralama: Standart</option>
        <option value="ucuz">Qiymət: Ucuzdan bahaya</option>
        <option value="baha">Qiymət: Bahadan ucuza</option>
        <option value="ad">Ada görə (A-Z)</option>
    </select>
</div>
<p class="tapilmadi" id="tapilmadi">😕 Belə bir məhsul tapılmadı.</p>
```

- `input type="text"` — mətn yazmaq üçün xana. `placeholder` — xana boş olanda görünən boz yazı.
- `oninput` — xanaya **hər hərf yazılanda** işə düşür (Enter gözləmir).
- `select` + `option` — açılan siyahı. `value` — JavaScript-in oxuduğu gizli dəyər; istifadəçi isə yazını görür.
- `onchange` — seçim dəyişəndə işə düşür.

2.7 Məhsul kartlarına edilən dəyişikliklər

HTML

```
<div class="mehsul-karti gorunme" data-qiymet="27" data-sira="1">
    <div class="mehsul-sekil-qutusu">
        <img src="Sezar.png" alt="Yuri Sezar" onclick="sekliBoyut(this)">
    </div>
    <h3>Yuri Sezar Qələm Qabı</h3>
    ...
    <button onclick="sebetaAt('Yuri Sezar Qələm Qabı', 27, 'Sezar.png')">Səbətə Əlavə Et</button>
</div>
```

- **`data-qiymet`**, **`data-sira`** — **data atributları**. Elementin içində əlavə məlumat saxlayırıq. JavaScript-də `kart.dataset.qiymet` ilə oxunur. Sıralama üçün lazımdır: `data-sira` — kartın ilkin yeri (standart sıraya qayıtmaq üçün).
- **`sebetaAt('...', 27, 'Sezar.png')`** — köhnədə funksiya parametrsiz idi və sadəcə rəqəmi artırırdı. İndi **3 parametr** ötürürük: ad, qiymət, şəkil. Beləcə funksiya *hansı* məhsulun əlavə olunduğunu bilir.
- **`sekliBoyut(this)`** — `this` "kliklənən elementin özü" deməkdir, yəni həmin `img`.

2.8 Əlaqə bölməsi və form

HTML

```
<div class="elaqe-kartlari">
    <a class="elaqe-karti" href="tel:+994558672327">
        <span class="elaqe-ikona">📞</span>
        <span><small>Zəng edin</small><b>055 867 23 27</b></span>
    </a>
    <a class="elaqe-karti" href="https://wa.me/994558672327" target="_blank">
        <span class="elaqe-ikona">💬</span>
        <span><small>WhatsApp</small><b>055 867 23 27</b></span>
    </a>
</div>
```

- `href="tel:+994558672327"` — **tel:** linki. Telefonda kliklәyəndə birbaşa zəng ekranı açılır. Nömrə beynəlxalq formatda yazılır: `+994` (Azərbaycan kodu) + əvvəlindəki 0 olmadan `55 867 23 27`.
- `https://wa.me/994558672327` — WhatsApp-ın rəsmi linki; bu nömrə ilə söhbət açır.
- `target="_blank"` — linki yeni vərəqdə (tab) açır ki, sayt bağlanmasın.
- Nömrə footer-ə də əlavə olunub.

HTML

```
<section class="elaqe-bolmesi gorunme" id="elaqe">
    <form class="elaqe-formu" onsubmit="formuGonder(event)">
        <input type="text" id="formAd" placeholder="Adınız">
        <input type="text" id="formEmail" placeholder="E-poçt ünvanınız">
        <textarea id="formMesaj" rows="5" placeholder="Mesajınız"></textarea>
        <p class="form-xeta" id="formXeta"></p>
        <button type="submit" id="formDuymesi">Mesajı Göndər</button>
    </form>
</section>
```

Köhnə saytda menyuda “Əlaqə” linki var idi, amma `id="elaqe"` olan bölmə yox idi — link heç yerə getmirdi. İndi var. `form` — məlumat toplamaq üçün konteynerdir. `button type="submit"` basılanda formun `onsubmit` hadisəsi baş verir. `event` — hadisə haqqında məlumatı funksiyaya ötürür (səhifənin yenilənməsini dayandırmaq üçün lazımdır). `textarea` — çox sətirli mətn xanası, `rows="5"` — 5 sətir hündürlüyü.

2.9 Footer, səbət paneli, modal, bildiriş, yuxarı düyməsi

HTML

```
<footer>
    <p>© <span id="il"></span> Jafil & Shavi — Premium 3D Studio</p>
</footer>

<div class="qara-fon" id="qaraFon" onclick="sebetiAcBagla()"></div>
<aside class="sebet-paneli" id="sebetPaneli">
    <div class="sebet-basliq">
        <h3>🛒 Səbətim</h3>
        <button class="bagla-duymesi" onclick="sebetiAcBagla()">✕</button>
    </div>
    <ul class="sebet-siyahisi" id="sebetSiyahisi"></ul>
    <div class="sebet-alt">
        <div class="cem-setri"><span>Cəmi:</span><span id="cemQiymet">0 AZN</span></div>
        <button class="sifaris-duymesi" onclick="sifarisVer()">Sifariş Ver</button>
        <button class="temizle-duymesi" onclick="sebetiTemizle()">Səbəti Təmizlə</button>
    </div>
</aside>

<div class="sekil-modal" id="sekilModal" onclick="modaliBagla()">
    <img id="modalSekil" src="" alt="Böyük şəkil">
    <p id="modalYazi"></p>
</div>

<div class="bildiris" id="bildiris"></div>

<button class="yuxari-duymesi" id="yuxariDuymesi" onclick="yuxariQalx()">↑</button>
```

- `footer` — səhifənin alt hissəsi üçün semantik teq. `<span id="il">` boşdur — ili JavaScript yazır.
- `aside` — əsas məzmundan kənar, "yan" məzmun üçün semantik teq (bizim səbət paneli).
- `ul` — sıralanmamış siyahı. İçi boşdur, çünki məhsulları JavaScript əlavə edir (`li` elementləri kimi).
- `qara-fon` — səbət açılanda arxa fonu tündləşdirən təbəqə; ona klikləyəndə səbət bağlanır.
- Bu elementlərin hamısı başlanğıcda **gizlidir** — CSS onları gizlədir, JavaScript lazım olanda klass əlavə edib göstərir.

3 CSS əlavələri

3.1 CSS dəyişənləri (custom properties) — ən vacib yenilik

```css
:root {
    --yasil: #1b4d2e;
    --fon: #f8fafc;
    --kart: #ffffff;
    --metn: #0f172a;
    --boz: #64748b;
}

body {
    background-color: var(--fon);
    color: var(--metn);
}
```

`--yasil` kimi adlar **CSS dəyişənləridir**. Rəngi bir yerdə yazırıq, sonra hər yerdə `var(--yasil)` ilə istifadə edirik. `:root` — səhifənin ən üst elementi (html), yəni dəyişənlər bütün səhifəyə aiddir.

Niyə faydalıdır? Əvvəl yaşıl rəng (`#1b4d2e`) faylda 15-dən çox yerdə təkrar yazılmışdı. Rəngi dəyişmək istəsən, hamısını bir-bir tapmalı idin. İndi yalnız `:root`-dakı **bir sətri** dəyişirsən və `var(--yasil)` istifadə edən **bütün elementlər avtomatik** yeni rəngi alır.

3.2 transition — yumşaq keçid

```css
body {
    transition: background-color 0.4s ease, color 0.4s ease;
}
```

`transition` — xassə dəyişəndə dəyişikliyin birdən yox, verilən müddətdə (0.4 saniyə) yumşaq olmasını təmin edir. `ease` — sürətin əyrisi (yavaş başla, sürətlən, yavaş bitir).

3.3 @keyframes — animasiyalar

CSS

```css
@keyframes titreme {
    0%   { transform: scale(1); }
    30%  { transform: scale(1.15) rotate(-4deg); }
    60%  { transform: scale(0.95) rotate(3deg); }
    100% { transform: scale(1); }
}

.sebet-qutusu.titre {
    animation: titreme 0.5s ease;
}

@keyframes yanibSon {
    0%, 100% { opacity: 1; }
    50%      { opacity: 0; }
}

.kursor {
    animation: yanibSon 0.8s infinite;
}
```

`@keyframes` — animasiyanın "kadrlarını" təsvir edir: 0%-də necə, 30%-də necə, 100%-də necə olsun. `animation: ad müddət` ilə elementə qoşulur. `infinite` — sonsuz təkrar.

- `transform: scale(1.15)` — 15% böyüt; `rotate(-4deg)` — 4 dərəcə fırlat.
- `.sebet-qutusu.titre` (arada boşluq yoxdur) — **hər iki klassı olan** element deməkdir. JS `titre` klassını əlavə edəndə animasiya oynayır.
- Saytda 3 animasiya var: **titreme** (səbət), **yanibSon** (kursor), **yuxariGel** (hero yazılarının aşağıdan gəlməsi).

3.4 Menyunun yumşaq açılması

CSS

```css
.menyu-paneli {
    opacity: 0;
    visibility: hidden;
    transform: translateY(-15px);
    transition: all 0.3s ease;
}

.menyu-paneli.aktiv {
    opacity: 1;
    visibility: visible;
    transform: translateY(0);
}
```

Köhnədə `display: none / block` istifadə olunurdu — menyu birdən görünürdü, çünki `display` animasiya olunmur. İndi `opacity` (şəffaflıq: 0 = görünməz, 1 = tam görünən) və `transform` animasiya olunur. `visibility: hidden` — görünməz menyuya təsadüfən kliklәməyin qarşısını alır.

3.5 Səbət paneli — position: fixed və translateX

CSS

```css
.sebet-paneli {
    position: fixed;
    top: 0;
    right: 0;
    width: 400px;
    max-width: 100%;
    height: 100%;
    z-index: 1600;
    display: flex;
    flex-direction: column;
    transform: translateX(100%);   /* başlanğıcda ekrandan kənarda */
    transition: transform 0.4s ease;
}

.sebet-paneli.acik {
    transform: translateX(0);
}

.qara-fon {
    position: fixed;
    inset: 0;
    background-color: rgba(0, 0, 0, 0.5);
}
```

- `position: fixed` — element ekrana “yapışır”, səhifə sürüşdürülsə də yerində qalır.
- `translateX(100%)` — paneli öz eni qədər sağa itələyir, yəni ekrandan çıxarır. `acik` klassı əlavə olunanda `translateX(0)` olur və panel sürüşərək gəlir.
- `z-index` — elementlərin üst-üstə düzülmə sırası. Böyük rəqəm = daha üstdə. Header 1000, səbət 1600, modal 2000, bildiriş 2500.
- `inset: 0` — `top/right/bottom/left: 0`-ın qısa yazılışı; element bütün ekranı tutur.
- `rgba(0,0,0,0.5)` — qara rəng, 50% şəffaflıqla.
- `max-width: 100%` — telefonda panel ekrandan enli olmasın.

3.6 Flexbox və Grid — yeni yerlərdə

```css
.alet-paneli { display: flex; gap: 15px; }
.alet-paneli input { flex: 1; }          /* qalan bütün yeri tut */

.sebet-siyahisi {
    flex: 1;
    overflow-y: auto;                    /* çox məhsul olanda sürüşdür */
}

.ustunlukler {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    margin: -60px auto 0 auto;           /* mənfi margin */
}
```

- `flex: 1` — flex konteynerində boş qalan yerin hamısını bu element tutur (axtarış xanası enli, select dar qalır).
- `overflow-y: auto` — məzmun sığmayanda şaquli sürüşdürmə zolağı çıxır.
- `repeat(3, 1fr)` — 3 bərabər sütun. `fr` = boş yerin bir hissəsi.
- **Mənfi margin** (`-60px`) — üstünlük kartlarını 60px yuxarı çəkir, hero şəklinin üstünə "çıxırlar". `z-index: 2` onları şəklin üstündə saxlayır.

3.7 Hover, focus və active effektləri

```css
.mehsul-karti:hover .mehsul-sekil-qutusu img {
    transform: scale(1.08);
}

.alet-paneli input:focus {
    border-color: var(--yasil);
    box-shadow: 0 0 0 4px rgba(27, 77, 46, 0.15);
}

button:active {
    transform: scale(0.95);
}
```

- `:hover` — siçan üstünə gələndə. Burada: **kartın** üstünə gələndə **içindəki şəkil** 8% böyüyür (valideyn:hover övlad).
- `:focus` — xanaya klikləyib yazmağa hazır olanda. `box-shadow` ətrafında yaşıl "halqa" yaradır.
- `:active` — düyməni basılı saxlayanda; düymə azca kiçilir, "basılma" hissi verir.

3.8 Sürüşdürmə animasiyası klassları

CSS

```css
.gorunme {
    opacity: 0;
    translate: 0 50px;
    transition: opacity 0.7s ease, translate 0.7s ease;
}

.gorunme.goruldu {
    opacity: 1;
    translate: 0 0;
}
```

Başlanğıcda element görünməz və 50px aşağıdadır. JavaScript onu ekranda görəndə `goruldu` klassı əlavə edir və element yumşaq şəkildə yuxarı qalxaraq görünür. **Niyə `transform` yox, `translate`?** Çünki məhsul kartının hover effekti artıq `transform` istifadə edir; eyni xassəni iki yerdə işlətsək, biri digərini pozardı. `translate` ayrıca xassədir — ikisi birlikdə işləyir.

3.9 Kənarlarda bitki naxışı — SVG və psevdo-elementlər

Naxışın özü `bitki.svg` faylındadır. **SVG** (Scalable Vector Graphics) — şəkli piksellərlə yox, **riyazi fiqurlarla** (xətt, əyri, dairə) təsvir edən formatdır. Ona görə istənilən ölçüdə kəskin qalır və çəkisi çox azdır (~3 KB). SVG də HTML kimi teqlərlə yazılır:

HTML

```html
<svg xmlns="http://www.w3.org/2000/svg" width="140" height="320">
    <defs>
        <g id="yarpaq">
            <path d="M0,0 C12,-14 34,-14 46,0 C34,14 12,14 0,0 Z" fill="#2f7a4a"/>
        </g>
    </defs>

    <!-- Gövdə: yuxarı və aşağı ucu eyni nöqtədədir (40,0) və (40,320) -->
    <path d="M40,0 C80,80 0,240 40,320" fill="none" stroke="#1b4d2e" stroke-width="3"/>

    <!-- Xarici g yarpağı yerinə qoyur, daxili use isə tərpənir -->
    <g transform="translate(51,41) rotate(-35)">
        <use href="#yarpaq">
            <animateTransform attributeName="transform" type="rotate"
                values="-12;12;-12" keyTimes="0;0.5;1"
                calcMode="spline" keySplines=".45 0 .55 1;.45 0 .55 1"
                dur="6s" begin="0s" repeatCount="indefinite"/>
        </use>
    </g>
    ... (daha 8 yarpaq və 3 çiçək, hər biri fərqli begin ilə)
</svg>
```

- `path d="..."` — fiqurun yolu: `M` — "bu nöqtəyə keç", `C` — əyri xətt (Bezier əyrisi), `Z` — fiquru bağla.

- `defs` + `g id="yarpaq"` — yarpağı **bir dəfə** təyin edirik, sonra `use href="#yarpaq"` ilə istədiyimiz qədər təkrar istifadə edirik. Bu, proqramlaşdırmadakı **funksiya** ideyasına bənzəyir.
- `transform="translate(51,41) rotate(-35)"` — yarpağı (51, 41) nöqtəsinə köçür və 35° fırlat.
- Gövdənin yuxarı və aşağı ucu eyni `x=40` nöqtəsindədir — şəkil alt-alta təkrarlananda sarmaşıq **qırılmadan** davam edir.

Yarpaqların tərpənməsi. Gövdə yerində qalır, yalnız yarpaq və çiçəklər yellənir:

- Hər yarpaq iki qatdır: **xarici** `g` onu gövdədəki yerinə qoyur (hərəkət etmir), **daxili** `use` isə fırlanır. Yarpağın dibi `(0,0)` nöqtəsindədir, ona görə yarpaq **dibindən** — gövdəyə bağlandığı yerdən fırlanır, real yarpaq kimi.
- `animateTransform` — SVG-nin öz animasiya teqi. `type="rotate"` — fırlanma; `values="-12;12;-12"` — −12°-dən +12°-yə, sonra geri; `dur="6s"` — bir dövr 6 saniyə; `repeatCount="indefinite"` — sonsuz.
- `calcMode="spline"` + `keySplines` — hərəkətin sürət əyrisi (CSS-dəki `ease-in-out` kimi): yavaş başlayır, ortada sürətlənir, yavaş bitir.
- `begin="-2.4s"` — mənfi gecikmə: animasiya sanki 2.4 saniyə əvvəl başlayıb. Hər yarpağa fərqli dəyər verilib ki, hamısı eyni anda yox, **növbə ilə** tərpənsin — külək effekti.
- **Niyə CSS animasiyası yox?** SVG fon şəkli kimi (`background-image`) istifadə olunanda içindəki CSS animasiyası brauzerdə işləmir, `animateTransform` isə işləyir.

CSS

```
body::before,
body::after {
    content: "";
    position: fixed;
    top: 0;
    bottom: 0;
    width: 140px;
    background-image: url("bitki.svg");
    background-repeat: repeat-y;
    background-size: 140px 320px;
    opacity: 0.55;
    pointer-events: none;
    z-index: -1;
}

body::before { left: 0; }

body::after {
    right: 0;
    transform: scaleX(-1);
    background-position: 0 160px;
}
```

- `::before` və `::after` — **psevdo-elementlər**. CSS elementin içində əlavə, "saxta" element yaradır — HTML-ə heç nə yazmadan. `::before` sol sarmaşıq, `::after` sağ sarmaşıqdır.
- `content: ""` — psevdo-elementin görünməsi üçün **mütləqdir** (boş olsa belə).
- `background-repeat: repeat-y` — şəkil yalnız **şaquli** (y oxu boyunca) təkrarlanır, sonsuz sarmaşıq alınır.

- `pointer-events: none` — naxış kliklərə mane olmur, siçan onun "içindən keçir".
- `z-index: -1` — naxış bütün məzmunun **arxasında** qalır, kartların üstünə çıxmır.
- `transform: scaleX(-1)` — üfüqi **güzgü əksi**: sağ tərəfdəki sarmaşıq solun əksidir.
- `background-position: 0 160px` — sağ naxış yarım şəkil aşağı sürüşdürülüb ki, iki tərəf eyni görünməsin.
- Planşetdə naxış nazikləşir (70px), telefonda isə `display: none` ilə gizlədilir — kiçik ekranda məzmuna yer qalsın.

3.10 Media queries — mobil uyğunluq

```css
@media (max-width: 650px) {
    .mehsullar-qutusu,
    .ustunlukler {
        grid-template-columns: 1fr;
    }
    .alet-paneli {
        flex-direction: column;
    }
    .sebet-qutusu {
        padding: 10px 14px;
        font-size: 14px;
    }
}
```

`@media (max-width: 650px)` — "ekranın eni 650px və ya daha azdırsa, bu qaydaları tətbiq et". Telefonda kartlar bir sütuna düşür, axtarış və sıralama alt-alta gəlir, header elementləri kiçilir. Bundan əlavə, `white-space: nowrap` səbət yazısının iki sətrə bölünməsinin qarşısını alır.

4 JavaScript — hər funksiyanın izahı

Bütün kod `script.js` faylındadır və 11 nömrələnmiş hissəyə bölünüb. Aşağıda hər funksiya: **nə edir**, **kodu**, **sətir-sətir izahı** və **istifadə olunan anlayışlar**.

`menyunuAcBagla()`

Hissə 1 · köhnədən qalıb

Nə edir? Menyu panelini açır, açıqdırsa bağlayır.

JavaScript

```
function menyunuAcBagla() {
    let menyu = document.getElementById("menyuPaneli");
    menyu.classList.toggle("aktiv");
}
```

Sətir-sətir izah

- `document.getElementById("menyuPaneli")` — HTML-də `id="menyuPaneli"` olan elementi tapır və `menyu` dəyişəninə yazır.
- `classList.toggle("aktiv")` — klass yoxdursa əlavə edir, varsa silir. "Açarı" kimi işləyir: bir klik açır, bir klik bağlayır.

DOM classList.toggle let

Səbət (Hissə 2) — məlumat strukturu

JavaScript

```
let sebet = [];
// Hər element belə bir obyektdir:
// { ad: "Bitki Qabı", qiymet: 30, sekil: "dibcek.png", say: 2 }
```

Köhnə versiyada sadəcə bir rəqəm (`let say = 0`) var idi — neçə dəfə klikləndiyini sayırdı, amma *nəyin* alındığını bilmirdi. **İndi** səbət **massivdir** (array — siyahı), içində hər məhsul bir **obyektdir** (object — açar: dəyər cütləri). Beləcə həm ad, həm qiymət, həm də say saxlanılır.

`sebetaAt(ad, qiymet, sekil)`

Hissə 2

Nə edir? Məhsulu səbətə əlavə edir. Məhsul artıq varsa, yeni sətir yaratmır — sadəcə sayını 1 artırır.

JavaScript

```
function sebetaAt(ad, qiymet, sekil) {
    let tapildi = false;

    for (let i = 0; i < sebet.length; i++) {
        if (sebet[i].ad === ad) {
            sebet[i].say = sebet[i].say + 1;
            tapildi = true;
        }
    }

    if (tapildi === false) {
        sebet.push({ ad: ad, qiymet: qiymet, sekil: sekil, say: 1 });
    }

    sebetiYenile();
    bildirisGoster("✅ " + ad + " səbətə əlavə olundu");

    let qutu = document.getElementById("sebetQutusu");
    qutu.classList.add("titre");
    setTimeout(function () {
        qutu.classList.remove("titre");
    }, 500);
}
```

Sətir-sətir izah

1. **Parametrlər**: `ad, qiymet, sekil` — HTML-dəki düymədən gəlir: `sebetaAt('Bitki Qabı', 30, 'dibcek.png')`.
2. `let tapildi = false` — "bayraq" dəyişəni (boolean). Əvvəlcə "tapılmadı" fərz edirik.
3. `for` dövrü səbətdəki hər məhsulu yoxlayır. `sebet.length` — massivdəki elementlərin sayı. `sebet[i]` — i-ci element.
4. `===` — ciddi bərabərlik (həm dəyər, həm tip eyni olmalıdır). Ad eynidirsə, say artırılır və bayraq `true` olur.
5. Dövr bitəndə hələ də `false`-dursa, `push()` ilə massivin sonuna yeni obyekt əlavə edirik.
6. `sebetiYenile()` — ekranı yeniləyir; `bildirisGoster()` — mesaj göstərir. (Funksiya başqa funksiyanı çağırır.)
7. `+` mətnlərlə işlədəndə onları birləşdirir (konkatenasiya): `"✅ " + ad + " ..."`.
8. Səbətə `titre` klassı əlavə olunur → CSS animasiyası oynayır. `setTimeout(funksiya, 500)` — 500 millisaniyə (0.5 san) sonra klassı silir ki, növbəti dəfə yenə oynaya bilsin.

parametr · massiv (array) · obyekt · for dövrü · if · boolean · push() · setTimeout

`sebetiYenile()`

Hissə 2

Nə edir? Səbət panelini sıfırdan yenidən çəkir: məhsul siyahısı, ümumi say (yuxarıdakı rəqəm) və cəm qiymət. Sonda səbəti brauzerin yaddaşına yazır.

JavaScript

```
function sebetiYenile() {
    let siyahi = document.getElementById("sebetSiyahisi");
    siyahi.innerHTML = "";

    let umumiSay = 0;
    let cem = 0;

    if (sebet.length === 0) {
        siyahi.innerHTML = '<li class="bos-sebet">Səbətiniz boşdur 🛍️</li>';
    }

    for (let i = 0; i < sebet.length; i++) {
        let mehsul = sebet[i];
        umumiSay = umumiSay + mehsul.say;
        cem = cem + mehsul.qiymet * mehsul.say;

        siyahi.innerHTML +=
            '<li class="sebet-mehsulu">' +
                '<img src="' + mehsul.sekil + '">' +
                '<h4>' + mehsul.ad + '</h4>' +
                '<button onclick="sayiDeyis(' + i + ', -1)">-</button>' +
                '<b>' + mehsul.say + '</b>' +
                '<button onclick="sayiDeyis(' + i + ', 1)">+</button>' +
                '<button onclick="mehsuluSil(' + i + ')">🗑</button>' +
            '</li>';
    }

    document.getElementById("sebet-sayi").innerText = umumiSay;
    document.getElementById("cemQiymet").innerText = cem + " AZN";

    localStorage.setItem("sebet", JSON.stringify(sebet));
}
```

Sətir-sətir izah

1. `innerHTML = ""` — siyahının içini təmizləyir (köhnə sətirlər təkrarlanmasın).
2. `umumiSay` və `cem` — **toplayıcı** (akkumulyator) dəyişənlər, 0-dan başlayır.
3. Səbət boşdursa, “Səbətiniz boşdur” yazısı qoyulur.
4. Dövrdə hər məhsul üçün: say toplanır, qiymət × say cəmə əlavə olunur.
5. `innerHTML +=` — mövcud HTML-in sonuna yeni `li` əlavə edir. HTML-i mətn kimi “yığırıq”.
6. Düymələrə `i` (index — məhsulun massivdəki nömrəsi) yazılır ki, hansı məhsulun dəyişəcəyi bilinsin. Məsələn: `sayiDeyis(0, -1)`.
7. `innerText` — elementin mətnini dəyişir (HTML kimi yox, düz yazı kimi).
8. `JSON.stringify(sebet)` — massivi mətnə çevirir, çünki `localStorage` yalnız mətn saxlaya bilir.

(Yuxarıdakı kod qısaldılıb; faylda şəkil, qiymət və say bloku üçün əlavə `div`-lər var.)

innerHTML innerText toplayıcı dəyişən index localStorage JSON.stringify

sayiDeyis(index, deyisme)

Hissə 2

Nə edir? Səbətdəki məhsulun sayını +1 və ya −1 dəyişir. Say 0-a düşsə, məhsulu silir.

JavaScript

```
function sayiDeyis(index, deyisme) {
    sebet[index].say = sebet[index].say + deyisme;

    if (sebet[index].say <= 0) {
        mehsuluSil(index);
    } else {
        sebetiYenile();
    }
}
```

Sətir-sətir izah

- `deyisme` ya `1`, ya `-1` olur. Mənfi ədədi toplamaq çıxmaq deməkdir — buna görə bir funksiya hər iki düyməyə xidmət edir.
- `if / else` — say 0 və ya daha azdırsa sil, əks halda sadəcə ekranı yenilə.

if / else müqayisə operatoru <=

mehsuluSil(index)

Hissə 2

Nə edir? Məhsulu səbətdən tamamilə çıxarır.

JavaScript

```
function mehsuluSil(index) {
    sebet.splice(index, 1);
    sebetiYenile();
}
```

Sətir-sətir izah

- `splice(index, 1)` — massivdən `index` mövqeyindən başlayaraq **1 element** silir. Məsələn `splice(2, 1)` — 3-cü elementi (sayma 0-dan başlayır) silir.

splice() index 0-dan başlayır

sebetiTemizle()

Hissə 2

Nə edir? Təsdiq soruşduqdan sonra bütün səbəti boşaldır.

JavaScript

```
function sebetiTemizle() {
    if (sebet.length === 0) {
        return;
    }
    if (confirm("Səbəti təmizləmək istədiyinizə əminsiniz?")) {
        sebet = [];
        sebetiYenile();
        bildirisGoster("🗑️ Səbət təmizləndi");
    }
}
```

Sətir-sətir izah

- `return` — funksiyadan dərhal çıxır. Səbət onsuz da boşdursa, heç nə etmirik.
- `confirm("...")` — "OK / Ləğv et" pəncərəsi açır. OK basılsa `true`, əks halda `false` qaytarır.
- `sebet = []` — dəyişənə yeni, boş massiv mənimsədirik.

return confirm()

sebetiAcBagla()

Hissə 2

Nə edir? Səbət panelini və arxadakı qara fonu açır/bağlayır.

JavaScript

```
function sebetiAcBagla() {
    document.getElementById("sebetPaneli").classList.toggle("acik");
    document.getElementById("qaraFon").classList.toggle("acik");
}
```

Sətir-sətir izah

Eyni anda iki elementə `acik` klassı əlavə olunur/silinir. CSS-də `.sebet-paneli.acik` paneli ekrana sürüşdürür, `.qara-fon.acik` fonu tündləşdirir. Bu funksiya 3 yerdən çağırılır: səbət düyməsi, ✕ düyməsi və qara fon.

classList.toggle

`sifarisVer()`

Hissə 2

Nə edir? Sifarişi "qəbul edir": ümumi məbləği göstərir, səbəti boşaldır və paneli bağlayır.

JavaScript

```
function sifarisVer() {
    if (sebet.length === 0) {
        bildirisGoster("⚠️ Səbətiniz boşdur!");
        return;
    }

    let cem = document.getElementById("cemQiymet").innerText;
    alert("Təşəkkür edirik! Sifarişiniz qəbul olundu.\nÜmumi məbləğ: " + cem);

    sebet = [];
    sebetiYenile();
    sebetiAcBagla();
}
```

Sətir-sətir izah

- Əvvəl **yoxlama**: boş səbətlə sifariş vermək olmaz.
- `alert()` — sadə məlumat pəncərəsi. `\n` — mətndə yeni sətir.
- Qeyd: bu tədris layihəsidir, sifariş real serverə göndərilmir — sadəcə simulyasiya edilir.

alert() \n yeni sətir

`bildirisGoster(metn)`

Hissə 3

Nə edir? Ekranın aşağısında 2.5 saniyəlik qısa mesaj (toast) göstərir.

JavaScript

```
let bildirisTaymeri;

function bildirisGoster(metn) {
    let bildiris = document.getElementById("bildiris");
    bildiris.innerText = metn;
    bildiris.classList.add("gorunur");

    clearTimeout(bildirisTaymeri);
    bildirisTaymeri = setTimeout(function () {
        bildiris.classList.remove("gorunur");
    }, 2500);
}
```

Sətir-sətir izah

- Mətn yazılır və `gorunur` klassı əlavə olunur → CSS mesajı aşağıdan qaldırır.
- `setTimeout` bir **nömrə** (taymer ID) qaytarır, onu `bildirisTaymeri`-də saxlayırıq.

- `clearTimeout` — köhnə taymeri ləğv edir. Niyə? Əgər tez-tez 3 məhsul əlavə etsən, birinci taymer ikinci mesajı vaxtından əvvəl gizlədərdi. Belə edəndə hər yeni mesaj tam 2.5 saniyə görünür.
- `bildirisTaymeri` funksiyadan **kənarda** elan olunub (qlobal dəyişən) ki, funksiya çağırışları arasında yadda qalsın.

setTimeout / clearTimeout | qlobal dəyişən | anonim funksiya

`mehsullariAxtar()`

Hissə 4

Nə edir? Axtarış xanasına yazılan sözü məhsul adlarında axtarır; uyğun gəlməyən kartları gizlədir.

JavaScript

```
function mehsullariAxtar() {
    let axtarilan = document.getElementById("axtarisXanasi").value.toLowerCase();
    let kartlar = document.querySelectorAll(".mehsul-karti");
    let tapilanSay = 0;

    for (let i = 0; i < kartlar.length; i++) {
        let ad = kartlar[i].querySelector("h3").innerText.toLowerCase();

        if (ad.includes(axtarilan)) {
            kartlar[i].style.display = "flex";
            tapilanSay++;
        } else {
            kartlar[i].style.display = "none";
        }
    }

    let yazi = document.getElementById("tapilmadi");
    if (tapilanSay === 0) {
        yazi.style.display = "block";
    } else {
        yazi.style.display = "none";
    }
}
```

Sətir-sətir izah

1. `.value` — input-a yazılan mətn. `toLowerCase()` — hamısını kiçik hərfə çevirir ki, “KİTAB” və “kitab” eyni sayılsın.
2. `querySelectorAll(".mehsul-karti")` — bu klassa malik **bütün** elementləri tapır (CSS seçicisi kimi yazılır).
3. `kartlar[i].querySelector("h3")` — kartın **içindəki** başlığı tapır.
4. `includes()` — mətnin içində başqa mətn varmı? Məs: `"harry potter kitab stendi".includes("kitab")` → `true`.
5. `style.display` — JavaScript ilə birbaşa CSS dəyişmək: `"none"` gizlədir, `"flex"` göstərir (kartlar flex konteynerdir).
6. `tapilanSay++` — `tapilanSay = tapilanSay + 1`-in qısa yazılışı.
7. Heç nə tapılmasa “Belə bir məhsul tapılmadı” yazısı göstərilir.

querySelectorAll | toLowerCase() | includes() | style.display | ++ operatoru

mehsullariSirala()

Hissə 5

Nə edir? Seçilmiş qaydaya görə (ucuz, baha, ad, standart) məhsul kartlarının yerini dəyişir.

JavaScript

```
function mehsullariSirala() {
    let secim = document.getElementById("siralamaSecimi").value;
    let qutu = document.getElementById("mehsullarQutusu");
    let kartlar = Array.from(qutu.querySelectorAll(".mehsul-karti"));

    kartlar.sort(function (a, b) {
        if (secim === "ucuz") {
            return a.dataset.qiymet - b.dataset.qiymet;
        } else if (secim === "baha") {
            return b.dataset.qiymet - a.dataset.qiymet;
        } else if (secim === "ad") {
            let adA = a.querySelector("h3").innerText;
            let adB = b.querySelector("h3").innerText;
            return adA.localeCompare(adB, "az");
        } else {
            return a.dataset.sira - b.dataset.sira;
        }
    });

    for (let i = 0; i < kartlar.length; i++) {
        qutu.appendChild(kartlar[i]);
    }
}
```

Sətir-sətir izah

1. `secim` — select-in seçilmiş `value`-su: "ucuz", "baha", "ad" və ya "standart".
2. `Array.from()` — `querySelectorAll` əsl massiv qaytarmır (NodeList), `sort()` isə yalnız massivdə işləyir. Ona görə çeviririk.
3. `sort(function (a, b) {...})` — **müqayisə funksiyası** iki kartı müqayisə edir:
   - mənfi ədəd qaytarsa → `a` əvvəl gəlir;
   - müsbət qaytarsa → `b` əvvəl gəlir;
   - 0 → yerləri dəyişmir.

   Məsələn: `10 - 27 = -17` → 10 AZN-lik əvvəl gəlir (ucuzdan bahaya).
4. `dataset.qiymet` — HTML-dəki `data-qiymet="27"`-ni oxuyur. Bu mətndir ("27"), amma `-` operatoru onu avtomatik ədədə çevirir.
5. `localeCompare(..., "az")` — mətnləri **Azərbaycan əlifbasına** görə müqayisə edir (Ç, Ə, Ü düzgün yerdə olur).
6. `appendChild()` — element artıq qutudadırsa, onu **sona köçürür**. Sıralanmış ardıcıllıqla hamısını köçürəndə kartlar yeni sırada düzülür.

Array.from | sort() | müqayisə funksiyası | dataset | localeCompare | appendChild

sekliBoyut(sekil) · modaliBagla()

Hissə 6

Nə edir? Məhsul şəklinə klikləyəndə onu ekranın ortasında böyük göstərir; istənilən yerə klikləyəndə bağlayır.

JavaScript

```
function sekliBoyut(sekil) {
    document.getElementById("modalSekil").src = sekil.src;
    document.getElementById("modalYazi").innerText = sekil.alt;
    document.getElementById("sekilModal").classList.add("acik");
}

function modaliBagla() {
    document.getElementById("sekilModal").classList.remove("acik");
}
```

Sətir-sətir izah

- `sekil` parametri — HTML-dəki `sekliBoyut(this)` -dan gələn kliklənmiş `img` elementidir.
- `sekil.src` — şəklin ünvanı; onu modal içindəki böyük şəklə köçürürük. `sekil.alt` — şəklin alt yazısı, başlıq kimi göstərilir.
- `classList.add / remove` — `toggle` -dan fərqli olaraq yalnız əlavə edir / yalnız silir.

this | src / alt xassələri | classList.add / remove

formuGonder(event) — 1-ci hissə: yoxlama

Hissə 7

Nə edir? Əlaqə formunu yoxlayır (validasiya): boş xanalar, e-poçt formatı, mesaj uzunluğu. Hər şey

düzdürsə, mesajı e-poçta göndərir (2-ci hissəyə bax).

JavaScript

```
const MAIL_UNVANI = "ceferliisrafil1@gmail.com";

function formuGonder(event) {
    event.preventDefault();

    let ad = document.getElementById("formAd").value.trim();
    let email = document.getElementById("formEmail").value.trim();
    let mesaj = document.getElementById("formMesaj").value.trim();
    let xeta = document.getElementById("formXeta");

    xeta.classList.remove("ugurlu");

    if (ad === "" || email === "" || mesaj === "") {
        xeta.innerText = "⚠️ Zəhmət olmasa bütün xanaları doldurun.";
        return;
    }

    if (!email.includes("@") || !email.includes(".")) {
        xeta.innerText = "⚠️ E-poçt ünvanı düzgün deyil.";
        return;
    }

    if (mesaj.length < 10) {
        xeta.innerText = "⚠️ Mesaj ən azı 10 simvol olmalıdır.";
        return;
    }

    // ... davamı: e-poçta göndərmə (növbəti səhifə)
```

Sətir-sətir izah

1. `const MAIL_UNVANI` — **sabit** (constant). `let`-dən fərqi: dəyəri sonradan dəyişdirilə bilməz. Mesajların gələcəyi e-poçt ünvanı burada bir dəfə yazılıb; ünvanı dəyişmək lazım olsa, yalnız bu sətri dəyişirsən. Ad böyük hərflə yazılır — sabitlər üçün qəbul olunmuş qaydadır.
2. `event.preventDefault()` — formun **standart davranışını** (səhifəni yeniləmək) dayandırır. Bunsuz səhifə yenilənər və mesajımız itərdi.
3. `trim()` — mətnin əvvəl və sonundakı boşluqları silir (yalnız boşluq yazıb "doldurmaq" olmasın).
4. `||` — **"VƏ YA"** məntiqi operatoru: üç şərtdən biri doğrudursa, xəta.
5. `!` — **"DEYİL"** operatoru: `!email.includes("@")` = "e-poçtda @ yoxdursa".
6. `mesaj.length` — mətnin simvol sayı.
7. Hər xətadan sonra `return` — funksiya dayanır, növbəti yoxlamalara keçmir. Buna **"erkən çıxış"** deyilir.

const | event | preventDefault | trim() | || (və ya) | ! (deyil) | length | validasiya

`formuGonder(event)` — 2-ci hissə: e-poçta göndərmə

Hissə 7

Nə edir? Yoxlamadan keçən mesajı **FormSubmit** xidməti vasitəsilə sayt sahibinin e-poçtuna (ceferliisrafil1@gmail.com) göndərir. Göndərilən vaxt düyməni bağlayır, nəticəyə görə uğur və ya xəta mesajı göstərir.

JavaScript

```
    let forma = event.target;
    let duyme = document.getElementById("formDuymesi");
    duyme.disabled = true;
    duyme.innerText = "Göndərilir...";

    fetch("https://formsubmit.co/ajax/" + MAIL_UNVANI, {
        method: "POST",
        headers: {
            "Content-Type": "application/json",
            "Accept": "application/json"
        },
        body: JSON.stringify({
            Ad: ad,
            email: email,
            Mesaj: mesaj,
            _subject: "Saytdan yeni mesaj: " + ad,
            _template: "table"
        })
    })
        .then(function (cavab) {
            return cavab.json();
        })
        .then(function (melumat) {
            if (melumat.success === "true" || melumat.success === true) {
                xeta.classList.add("ugurlu");
                xeta.innerText = "✅ Təşəkkürlər, " + ad + "! Mesajınız göndərildi.";
                forma.reset();
            } else {
                xeta.innerText = "⚠️ Mesaj göndərilmədi: " + melumat.message;
            }
        })
        .catch(function () {
            xeta.innerText = "⚠️ İnternet bağlantısını yoxlayın və yenidən cəhd edin.";
        })
        .finally(function () {
            duyme.disabled = false;
            duyme.innerText = "Mesajı Göndər";
        });
}
```

Sətir-sətir izah

Problem: HTML, CSS və JavaScript yalnız istifadəçinin brauzerində işləyir — onlar özləri e-poçt göndərə bilmir. E-poçt göndərmək üçün **server** lazımdır. Biz pulsuz **FormSubmit** (formsubmit.co) xidmətindən istifadə edirik: məlumatı ona göndəririk, o da bizim adımızdan e-poçt yazır.

1. `duyme.disabled = true` — düyməni bağlayır ki, istifadəçi təkrar basıb eyni mesajı iki dəfə göndərməsin. Yazı “Göndərilir...” olur.
2. `fetch(ünvan, parametrlər)` — JavaScript-in **internetə sorğu göndərmək** üçün funksiyası. Ünvan `"https://formsubmit.co/ajax/" + MAIL_UNVANI` — mesajın hara çatacağını göstərir.
3. `method: "POST"` — məlumat **göndəririk** (GET — almaq, POST — göndərmək).
4. `headers` — sorğu haqqında əlavə məlumat: “sənə JSON göndərirəm və cavabı da JSON kimi istəyirəm”.
5. `body: JSON.stringify({...})` — göndərilən məlumat: ad, e-poçt, mesaj. `_subject` — məktubun mövzusu, `_template: "table"` — məktub cədvəl şəklində səliqəli gəlsin. `email` sahəsi sayəsində məktuba “Cavab ver” basanda birbaşa yazan şəxsə cavab gedir.
6. `.then()` — `fetch` cavabı dərhal qaytarmır (internet vaxt aparır). Bu, **asinxron** əməliyyatdır: “cavab gələndə bunu et” deyirik. Birinci `then` cavabı JSON-a çevirir, ikincisi nəticəni yoxlayır.
7. `melumat.success` — xidmət “uğurlu” deyirsə, yaşıl təşəkkür mesajı göstərilir və form təmizlənir. Əks halda xidmətin xəta mesajı göstərilir.
8. `.catch()` — internet yoxdursa və ya sorğu alınmasa işləyir.
9. `.finally()` — nəticədən asılı olmayaraq **həmişə** işləyir: düyməni yenidən açırıq.

> **Vacib:** FormSubmit-ə ilk dəfə mesaj göndəriləndə o, **ceferliisrafil1@gmail.com** ünvanına “Activate Form” adlı təsdiq məktubu göndərir. Həmin məktubdakı düyməyə bir dəfə basmaq lazımdır — bundan sonra bütün mesajlar birbaşa poçta gəlir. Form sayt **internetdə** (məsələn, GitHub Pages-də) açılanda işləyir; faylı kompüterdən iki kliklə açanda (`file://`) xidmət sorğunu qəbul etməyə bilər.

fetch() POST JSON asinxron .then / .catch / .finally disabled

`yaziEffekti()`

Hissə 8

Nə edir? Hero bölməsində sözləri hərf-hərf yazır, gözləyir, hərf-hərf silir və növbəti sözə keçir. Sonsuz

davam edir.

JavaScript

```
let sozler = ["Qələm qabları", "Kitab stendləri", "Bitki qabları", "Fərdi 3D dizaynlar"];
let sozIndex = 0;
let herfIndex = 0;
let silinir = false;

function yaziEffekti() {
    let soz = sozler[sozIndex];
    let sahe = document.getElementById("yaziSahesi");

    if (silinir === false) {
        herfIndex++;
    } else {
        herfIndex--;
    }

    sahe.innerText = soz.substring(0, herfIndex);

    let gozleme = 100;

    if (silinir === false && herfIndex === soz.length) {
        silinir = true;
        gozleme = 1500;
    } else if (silinir === true && herfIndex === 0) {
        silinir = false;
        sozIndex = (sozIndex + 1) % sozler.length;
        gozleme = 400;
    }

    setTimeout(yaziEffekti, gozleme);
}
```

Sətir-sətir izah

1. **Vəziyyət dəyişənləri**: hansı söz (`sozIndex`), neçənci hərf (`herfIndex`), yazırıq yoxsa silirik (`silinir`).
2. Hər çağırışda hərf sayı 1 artır (yazanda) və ya 1 azalır (silərkən).
3. `substring(0, herfIndex)` — sözün əvvəlindən `herfIndex` qədər hərf götürür. Məs: `"Bitki".substring(0, 3)` → `"Bit"` .
4. `&&` — **"VƏ"** operatoru: hər iki şərt doğru olmalıdır.
5. Söz tam yazılanda → silməyə keçir və 1.5 saniyə gözləyir. Tam silinəndə → növbəti sözə keçir.
6. `%` — **qalıq** operatoru. `(3 + 1) % 4 = 0` — sonuncu sözdən sonra yenidən birinci sözə qayıdırıq.
7. `setTimeout(yaziEffekti, gozleme)` — funksiya **özünü yenidən çağırır** (müəyyən gecikmə ilə). Bu, sonsuz animasiya dövrü yaradır.

massiv | substring() | && (və) | % qalıq | setTimeout ilə təkrar

elementleriGoster() · scroll hadisəsi · yuxariQalx()

Hissə 9

Nə edir? Səhifə sürüşdürüləndə: (1) ekrana girən elementləri animasiya ilə göstərir, (2) "yuxarı qalx" düyməsini göstərir/gizlədir, (3) header-i kiçildir.

JavaScript

```
function elementleriGoster() {
    let elementler = document.querySelectorAll(".gorunme");

    for (let i = 0; i < elementler.length; i++) {
        let yuxariMesafe = elementler[i].getBoundingClientRect().top;

        if (yuxariMesafe < window.innerHeight - 80) {
            elementler[i].classList.add("goruldu");
        }
    }
}

window.addEventListener("scroll", function () {
    let duyme = document.getElementById("yuxariDuymesi");
    if (window.scrollY > 400) {
        duyme.classList.add("gorunur");
    } else {
        duyme.classList.remove("gorunur");
    }

    let header = document.querySelector("header");
    if (window.scrollY > 50) {
        header.classList.add("kicik");
    } else {
        header.classList.remove("kicik");
    }

    elementleriGoster();
});

function yuxariQalx() {
    window.scrollTo({ top: 0, behavior: "smooth" });
}
```

Sətir-sətir izah

- `getBoundingClientRect().top` — elementin **ekranın yuxarısından** məsafəsi (piksellə).
- `window.innerHeight` — brauzer pəncərəsinin hündürlüyü. Element ekranın aşağı kənarından 80px yuxarı girəndə `goruldu` klassı alır.
- `addEventListener("scroll", funksiya)` — **hadisə dinləyicisi**: hər dəfə səhifə sürüşdürüləndə funksiya işləyir. Bu, HTML-dəki `onclick` -in JavaScript-də yazılan alternatividir.
- `window.scrollY` — səhifənin nə qədər aşağı sürüşdürüldüyü (piksellə).
- `scrollTo({ top: 0, behavior: "smooth" })` — səhifənin ən başına yumşaq hərəkətlə qayıdır. Buraya obyekt ötürülür.

addEventListener | scroll hadisəsi | getBoundingClientRect | scrollY | scrollTo

Escape düyməsi (keydown)

Hissə 10

Nə edir? Klaviaturada Esc basılanda açıq olan modal, səbət və menyunu bağlayır.

JavaScript

```
document.addEventListener("keydown", function (event) {
    if (event.key === "Escape") {
        modaliBagla();
        document.getElementById("sebetPaneli").classList.remove("acik");
        document.getElementById("qaraFon").classList.remove("acik");
        document.getElementById("menyuPaneli").classList.remove("aktiv");
    }
});
```

Sətir-sətir izah

- `keydown` — klaviaturada düymə basılanda baş verən hadisə.
- `event.key` — hansı düymənin basıldığı (“Escape”, “Enter”, “a” və s.).
- Burada `remove` istifadə olunur, `toggle` yox — çünki məqsəd yalnız **bağlamaq**dır; bağlı pəncərəni təsadüfən açmaq istəmirik.

keydown event.key

Səhifə açılanda işləyən kod

Hissə 11

Nə edir? Sayt yüklənəndə bir dəfə işləyir: yadda saxlanmış səbəti bərpa edir, ili yazır, animasiyaları başladır.

JavaScript

```
let saxlanmisSebet = localStorage.getItem("sebet");
if (saxlanmisSebet !== null) {
    sebet = JSON.parse(saxlanmisSebet);
}
sebetiYenile();

document.getElementById("il").innerText = new Date().getFullYear();

yaziEffekti();
elementleriGoster();
```

Sətir-sətir izah

- `localStorage.getItem("sebet")` — əvvəl saxlanmış mətni oxuyur. Heç nə yoxdursa `null` qaytarır.
- `!==` — “ciddi bərabər deyil”.
- `JSON.parse()` — mətni yenidən massivə çevirir (`JSON.stringify` -ın əksi).
- `new Date().getFullYear()` — bu günün tarixindən ili götürür (2026). Footer hər il avtomatik yenilənir.

- Sonda `yaziEffekti()` animasiyanı başladır, `elementleriGoster()` isə ilk ekrandakı elementləri dərhal göstərir (sürüşdürməyi gözləmədən).

localStorage.getItem JSON.parse null Date obyekti

5 Əsas anlayışlar lüğəti

Anlayış	İzah
Dəyişən (let)	Məlumat saxlayan "qutu". `let say = 0;`
Sabit (const)	Dəyəri dəyişməyən "qutu". `const MAIL_UNVANI = "...";`
fetch()	İnternetə (serverə) sorğu göndərən funksiya.
POST sorğusu	Serverə məlumat **göndərmək** üçün sorğu növü (GET — almaq üçün).
Asinxron	Nəticəsi dərhal yox, bir az sonra gələn əməliyyat. `.then()` ilə gözlənilir.
tel: linki	Kliklәyəndə telefonda zəng açan link. `href="tel:+994..."`
Funksiya	Adı olan kod bloku; çağıranda işləyir. `function ad() { ... }`
Parametr	Funksiyaya ötürülən dəyər. `sebetaAt(ad, qiymet, sekil)`
Massiv (array)	Sıralı siyahı. `[1, 2, 3]` . Elementlər 0-dan nömrələnir.
Obyekt (object)	Açar–dəyər cütləri. `{ ad: "Bitki Qabı", qiymet: 30 }`
for dövrü	Kodu təkrar-təkrar icra edir. `for (let i = 0; i < 5; i++)`
if / else	Şərtə görə qərar vermə.
Boolean	Yalnız iki dəyər: `true` (doğru) və `false` (yanlış).
DOM	Document Object Model — HTML-in JavaScript-dəki "ağac" modeli. JS bununla səhifəni dəyişir.
getElementById	id-yə görə bir element tapır.
querySelector(All)	CSS seçicisinə görə element(lər) tapır.
classList	Elementin klasslarını idarə edir: add, remove, toggle, contains.
innerText / innerHTML	Elementin mətni / HTML məzmunu.
Hadisə (event)	İstifadəçinin hərəkəti: klik, yazma, sürüşdürmə, düymə basma.
addEventListener	Hadisəni JS-dən dinləmək üçün metod.
setTimeout	Kodu müəyyən millisaniyə sonra icra edir.
localStorage	Brauzerdə daimi yaddaş (səhifə bağlansa belə qalır). Yalnız mətn saxlayır.
JSON	Məlumatı mətn formatında yazma üsulu. `stringify` → mətnə, `parse` → geri.
CSS dəyişəni	`--ad: dəyər` ilə elan, `var(--ad)` ilə istifadə.
transition	Xassə dəyişəndə yumşaq keçid.
@keyframes	Addım-addım animasiya təsviri.

transform	Elementi hərəkət etdirmək, böyütmək, fırlatmaq.
position: fixed	Elementi ekrana bərkidir, sürüşdürmədə yerində qalır.
z-index	Elementlərin üst-üstə düzülmə sırası.
SVG	Vektor şəkil formatı: fiqurlar riyazi təsvir olunur, böyüdəndə keyfiyyət itmir.
Psevdo-element	`::before` / `::after` — CSS-in HTML-siz yaratdığı əlavə element.
Media query	Ekran ölçüsünə görə fərqli CSS qaydaları.
Semantik teq	Mənasını bildirən teq: header, section, main, aside, footer, nav.
data-* atributu	HTML elementində əlavə məlumat saxlamaq; JS-də `dataset` ilə oxunur.

6 Müəllimin verə biləcəyi suallar və cavablar

S: Səbət sayının yanında məhsulun adı və qiyməti necə yadda qalır?

Səbət `sebet` adlı massivdir. Hər məhsul obyekt kimi saxlanılır: `{ad, qiymet, sekil, say}`. Düymə `sebetaAt('ad', qiymət, 'şəkil')` funksiyasını parametrlərlə çağırır.

S: Eyni məhsulu iki dəfə əlavə etsəm, niyə iki sətir yaranmır?

`sebetaAt` əvvəlcə `for` dövrü ilə massivdə eyni adlı məhsul axtarır. Tapılsa, yalnız `say`-ı 1 artırır. Tapılmasa (`tapildi === false`), `push` ilə yeni obyekt əlavə edir.

S: Cəm qiymət necə hesablanır?

`sebetiYenile` funksiyasında `cem = 0`-dan başlayır və dövrdə hər məhsul üçün `qiymet * say` əlavə olunur.

S: Səhifəni yeniləyəndə səbət niyə itmir?

Hər dəyişiklikdən sonra səbət `JSON.stringify` ilə mətnə çevrilib `localStorage`-a yazılır. Səhifə açılanda `localStorage.getItem` ilə oxunub `JSON.parse` ilə yenidən massivə çevrilir.

S: localStorage ilə adi dəyişənin fərqi nədir?

Adi dəyişən səhifə yenilənəndə silinir. `localStorage` isə brauzerdə qalır, hətta brauzer bağlansa belə. Amma yalnız mətn saxlayır, ona görə JSON lazımdır.

S: Formdan yazılan mesaj e-poçtunuza necə gəlir?

JavaScript özü e-poçt göndərə bilmir, buna server lazımdır. `fetch()` ilə məlumatı (ad, e-poçt, mesaj) JSON formatında pulsuz **FormSubmit** xidmətinə `POST` sorğusu ilə göndəririk, o da mesajı `MAIL_UNVANI` sabitində yazılmış ünvana məktub kimi çatdırır.

S: fetch-dən sonra niyə .then() yazılır?

İnternet sorğusu vaxt aparır, cavab dərhal gəlmir — bu **asinxron** əməliyyatdır. `.then()` "cavab gələndə bunu et" deməkdir. `.catch()` xəta olanda, `.finally()` isə hər halda işləyir (düyməni yenidən açmaq üçün).

S: let ilə const-un fərqi nədir?

`let` ilə yaradılan dəyişənin dəyəri sonradan dəyişə bilər (məs. `sebet`). `const` isə sabitdir — dəyəri dəyişdirilə bilməz. E-poçt ünvanı dəyişmədiyi üçün `const MAIL_UNVANI` yazılıb.

S: Kənarlardakı bitki naxışı necə edilib? HTML-də heç nə görünmür.

Naxış `bitki.svg` faylında vektor şəkil kimi çəkilib. CSS-də `body::before` və `body::after` psevdo-elementləri yaradılıb — onlar HTML-ə yazılmır, CSS özü yaradır. `position: fixed` ilə kənarlara bərkidilib, `repeat-y` ilə şaquli təkrarlanır, `z-index: -1` ilə məzmunun arxasındadır, sağ tərəf isə `scaleX(-1)` ilə güzgü əksidir. Yarpaqlar SVG-nin içindəki `animateTransform` teqi ilə öz dibi ətrafında fırlanaraq tərpənir, gövdə isə yerində qalır.

S: SVG ilə PNG şəklin fərqi nədir?

PNG piksellərdən ibarətdir — böyüdəndə bulanıqlaşır. SVG isə xətt və əyrilərin riyazi təsviridir — istənilən ölçüdə kəskin qalır, çəkisi də çox kiçikdir. SVG mətn faylıdır, teqlərlə yazılır (HTML kimi).

S: Fontları niyə dəyişdiniz?

Köhnə fontlarda (Syne, Plus Jakarta Sans) “ə” hərfinin qlifi yox idi, brauzer onu başqa fontdan götürürdü və hərf yad görünürdü. Inter və Montserrat bütün Azərbaycan hərflərini dəstəkləyir.

S: Telefon nömrəsinə klikləyəndə niyə zəng açılır?

Link `href="tel:+994558672327"` şəklində yazılıb. `tel:` protokolu telefonda zəng proqramını açır. `https://wa.me/994558672327` isə WhatsApp söhbətini açır.

S: Axtarış böyük-kiçik hərfə həssasdırmı?

Xeyr. Həm axtarılan söz, həm məhsul adı `toLowerCase()` ilə kiçik hərfə çevrilir, sonra `includes()` ilə müqayisə olunur.

S: Sıralamada sort() içindəki funksiya nə qaytarır?

Mənfi ədəd — birinci element əvvəl gəlir; müsbət — ikinci element əvvəl; 0 — yer dəyişmir. `a - b` artan, `b - a` azalan sıra verir.

S: Niyə Array.from istifadə etdiniz?

`querySelectorAll` NodeList qaytarır, onda `sort()` metodu yoxdur. `Array.from` onu əsl massivə çevirir.

S: data-qiymet nədir, niyə qiyməti h3-dən oxumadınız?

`data-*` atributu elementdə əlavə məlumat saxlamağın standart üsuludur. “27 AZN” mətnindən rəqəmi ayırmaq əvəzinə təmiz rəqəmi `dataset.qiymet` ilə birbaşa oxuyuruq — daha sadə və etibarlıdır.

S: event.preventDefault() nə edir?

Formun standart davranışını — göndərib səhifəni yeniləməyi — dayandırır. Belə olmasa, səhifə yenilənər və yoxlama mesajı görünməzdi.

S: Formda hansı yoxlamalar var?

1) Xanalar boş olmamalıdır (`trim()` ilə boşluqlar da nəzərə alınır); 2) e-poçtda "@" və "." olmalıdır; 3) mesaj ən azı 10 simvol olmalıdır. Hər xətada `return` ilə funksiyadan çıxılır.

S: Yazı effekti necə sonsuz işləyir? Dövr (loop) yoxdur axı.

`yaziEffekti` sonunda `setTimeout(yaziEffekti, gozleme)` ilə özünü yenidən çağırır. Hər çağırış bir hərf əlavə edir/silir. `%` operatoru sonuncu sözdən sonra birinciyə qaytarır.

S: onclick ilə addEventListener-in fərqi nədir?

`onclick` HTML-də atribut kimi yazılır. `addEventListener` isə JS-də yazılır və eyni hadisəyə bir neçə funksiya qoşmağa imkan verir. Saytda hər ikisi var: düymələr üçün `onclick`, scroll və klaviatura üçün `addEventListener`.

S: toggle, add və remove arasında fərq?

`add` — klassı əlavə edir; `remove` — silir; `toggle` — varsa silir, yoxdursa əlavə edir. Açıb-bağlama üçün `toggle`, yalnız bağlamaq üçün (Escape) `remove` istifadə olunub.

S: Səbət paneli ekrandan necə "sürüşüb" gəlir?

CSS-də panel `position: fixed` və `transform: translateX(100%)` ilə ekranın sağından kənardadır. JS `acik` klassı əlavə edəndə `translateX(0)` olur, `transition` bunu 0.4 saniyəlik animasiyaya çevirir.

S: Menyu üçün niyə display: none yox, opacity istifadə etdiniz?

`display` xassəsi animasiya olunmur — element birdən görünür. `opacity` və `transform` isə yumşaq animasiya olunur. `visibility: hidden` gizli menyuya kliklәməyin qarşısını alır.

S: Loqonun yolunu niyə dəyişdiniz?

Köhnə yol kompüterə bağlı mütləq yol idi (`C:\Users\...`). Sayt başqa yerdə açılanda loqo görünməzdi. Nisbi yol `jafil.jpeg` hər yerdə işləyir.

S: z-index nədir və saytda necə istifadə olunub?

Elementlərin üst-üstə düzülmə sırasıdır. Böyük rəqəm daha üstdədir: header (1000) < səbət fonu (1500) < səbət paneli (1600) < şəkil modalı (2000) < bildiriş (2500).

S: Sayt telefonda necə uyğunlaşır?

`@media (max-width: 650px)` qaydaları ilə: kartlar 1 sütuna düşür, axtarış paneli alt-alta düzülür, header elementləri kiçilir. Səbət panelində `max-width: 100%` var ki, ekrandan enli olmasın.

Uğurlar! 🎓 Hər funksiyanı saytda özün sına — düyməni bas, kodda nəyin işlədiyini izlə. Ən yaxşı öyrənmə yolu budur.